

Calcolatori Elettronici T
Ingegneria Informatica

ISA DLX: implementazione *pipelined*



Principio del *Pipelining*

Il *pipelining* è oggi la principale tecnica di base impiegata per rendere “veloce” una CPU .

L'idea alla base del *pipelining* è generale, e trova applicazione in molteplici settori dell'industria (linee di produzione, oleodotti ...)

Un sistema, S , deve eseguire N volte un'attività A :

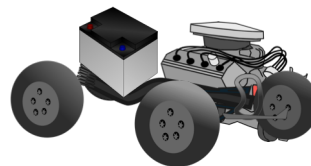
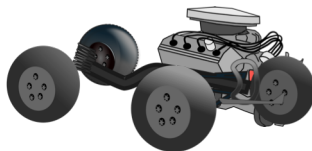


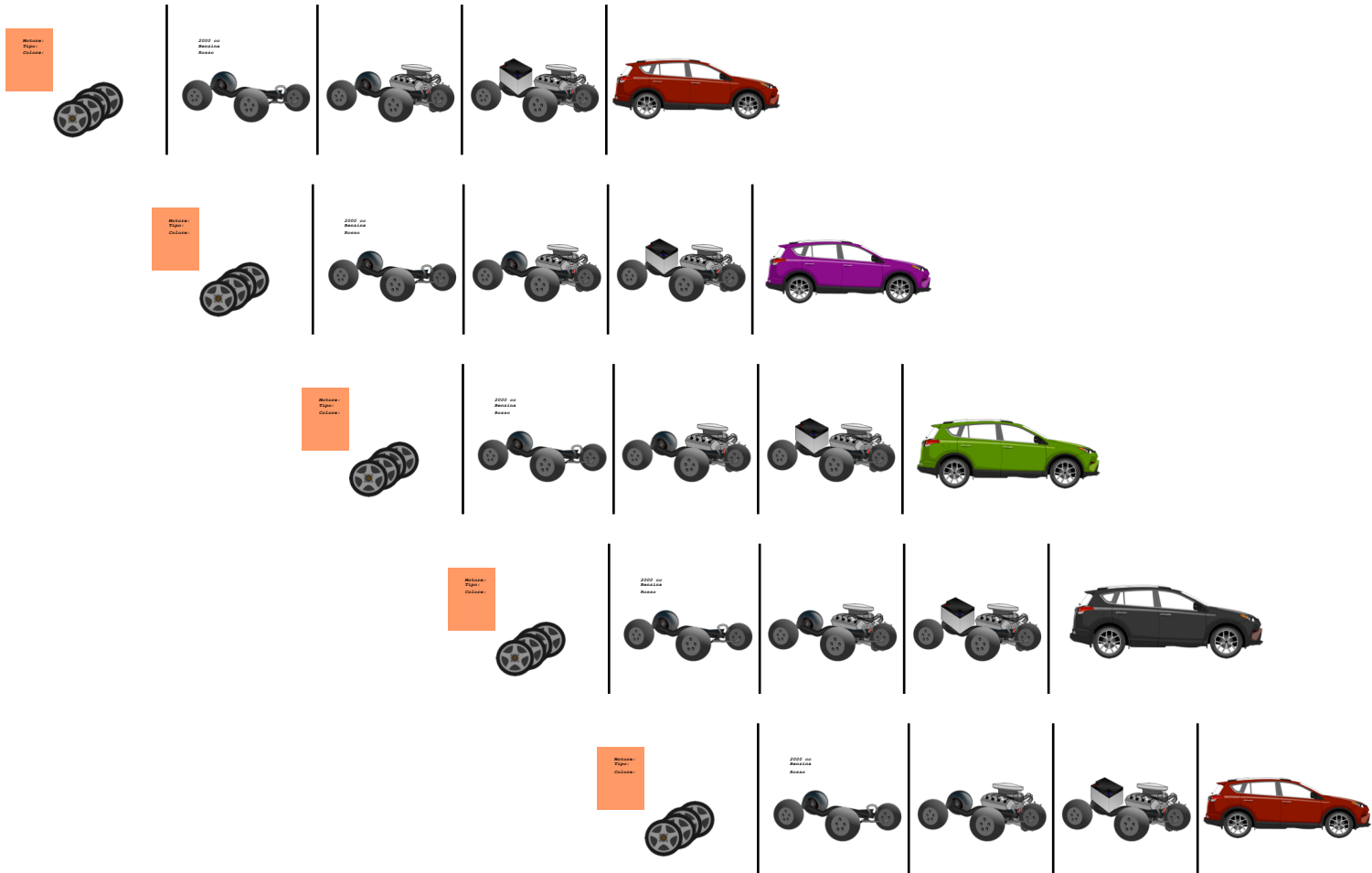
Latency : tempo che intercorre fra l'inizio ed il completamento dell'attività A (T_A).

Throughput : frequenza con cui vengono completate le attività.



Motore: 2000 cc
Tipo: Benzina
Colore: Rosso



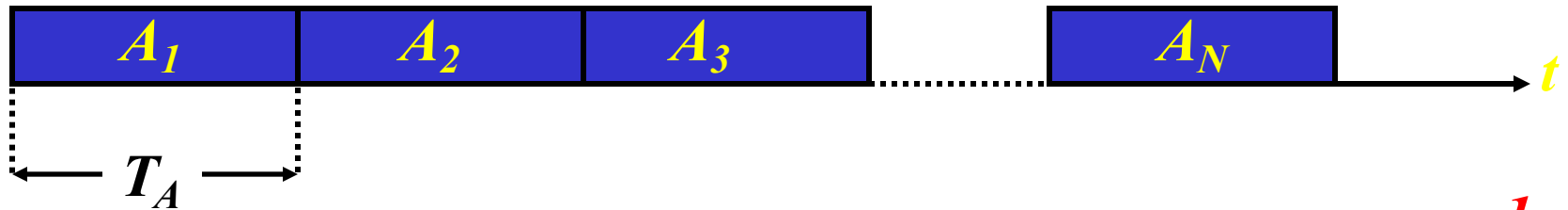




- ***Latenza***: 5 fasi (clock)
- ***Throughput***: a regime, dopo 5 fasi (clock), un'automobile per fase (clock)

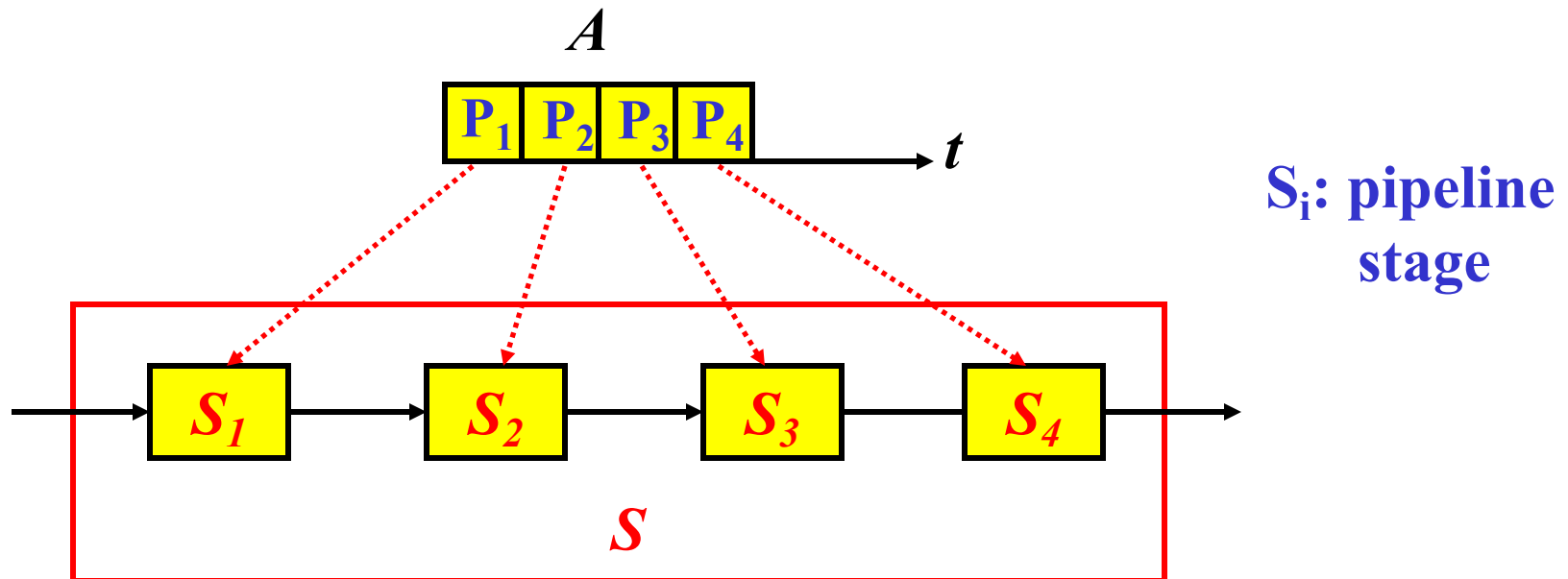
Principio del *Pipelining*

1) Sistema Sequenziale

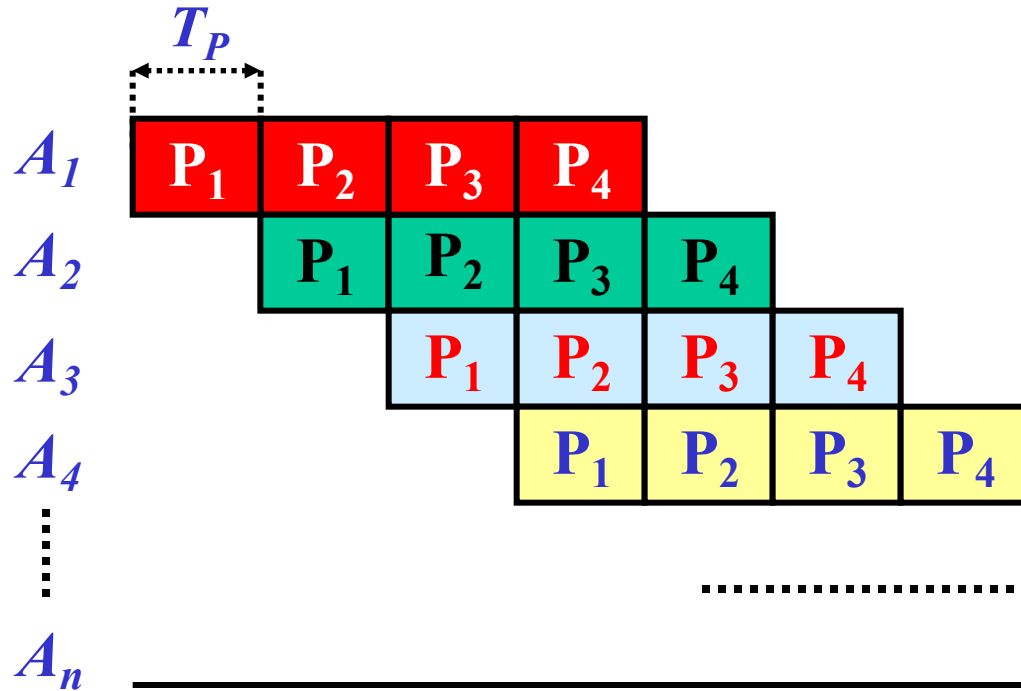


Latency (tempo di esecuzione di una istruzione) = T_A *Throughput = $\frac{1}{T_A}$*

2) Sistema in Pipeline



Principio del *Pipelining*

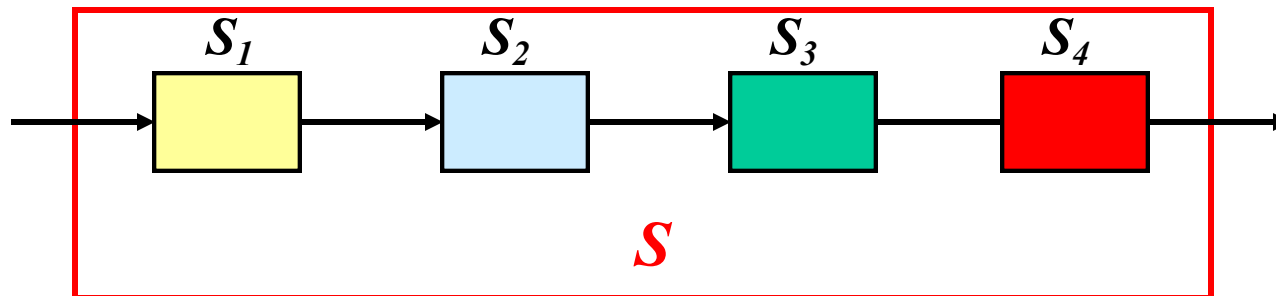


T_P : pipeline cycle

$$\text{Latency}(2) = 4 * T_P = T_A$$

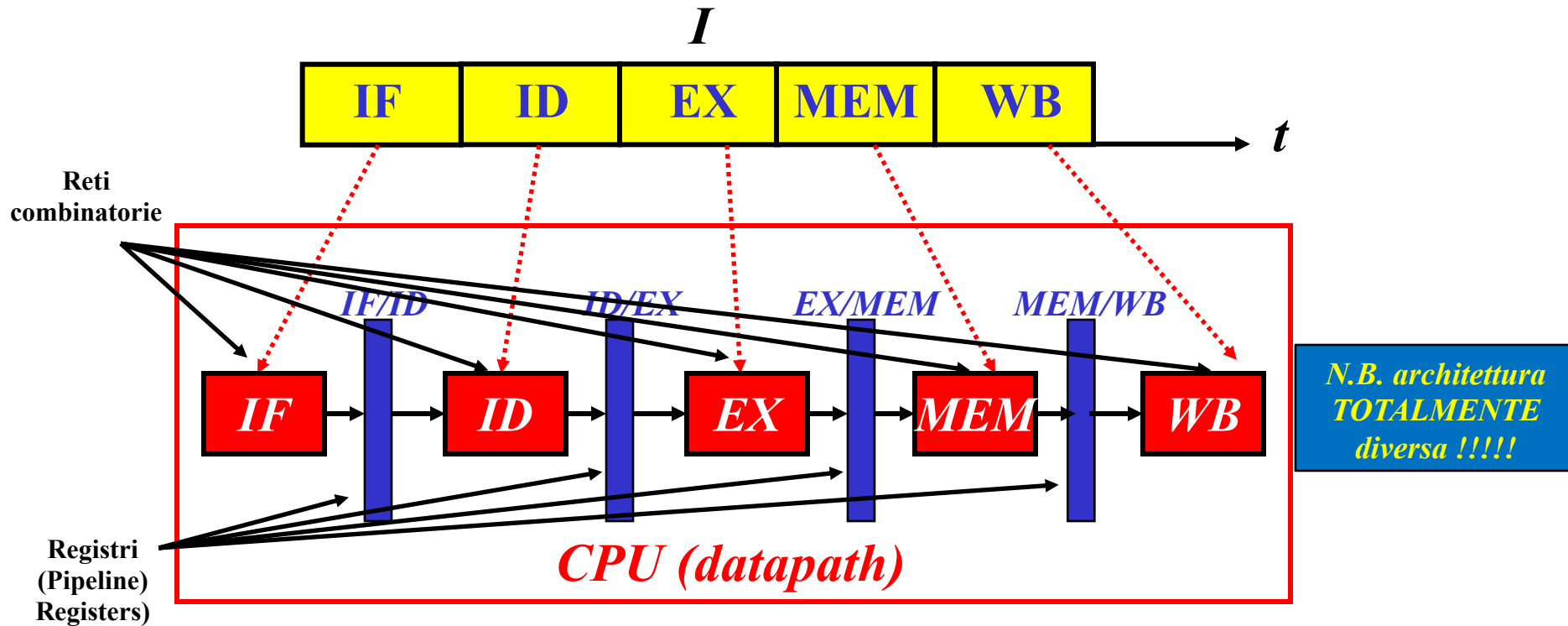
$$\text{Throughput}(2) \cong \frac{1}{T_P} = \frac{4}{T_A}$$

$$= 4 * \text{Throughput}(1)$$



Pipelining in una CPU (DLX)

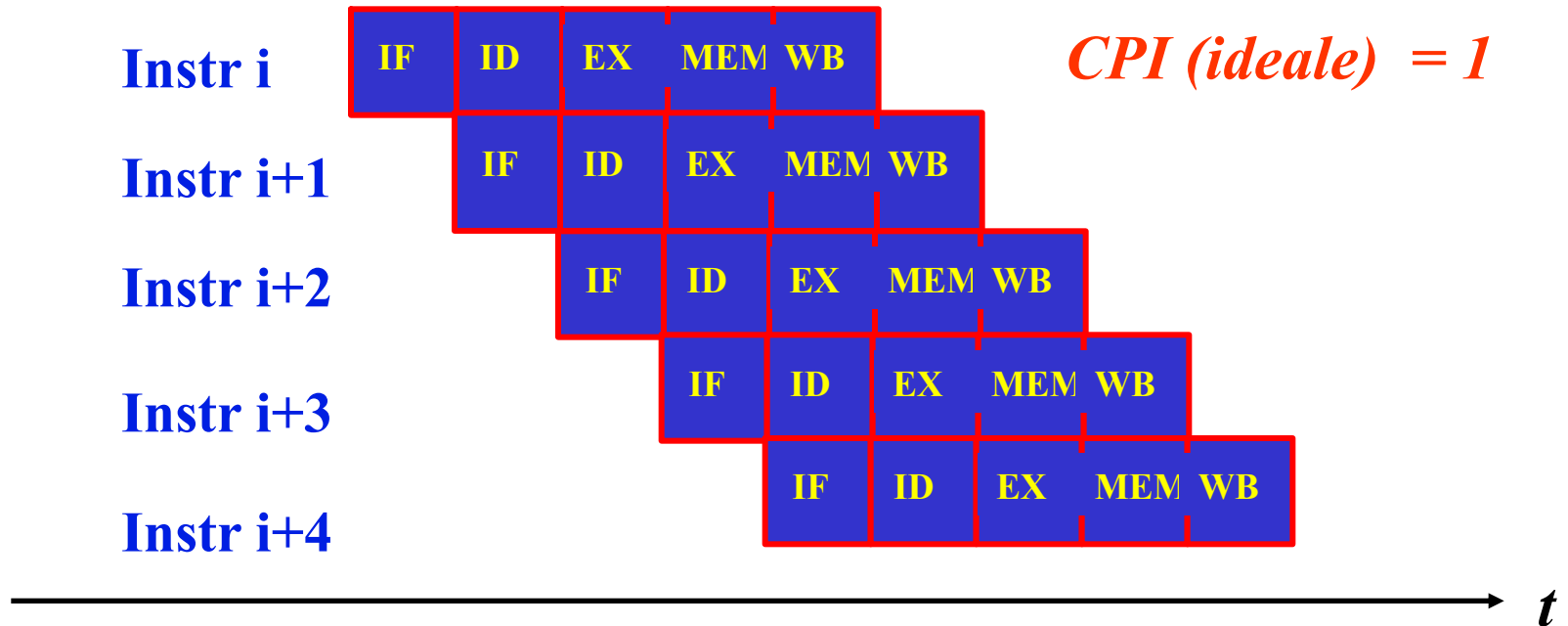
Attività: $A_1, A_2, A_3 \dots A_N$  *Istruzioni:* $I_1, I_2, I_3 \dots I_N$



Pipeline Cycle $\longrightarrow$ *Clock Cycle* $\longrightarrow$ *Ritardo dello stadio più lento*

$\downarrow$
CPI=1 (idealmente !)

Pipeline del *DLX*

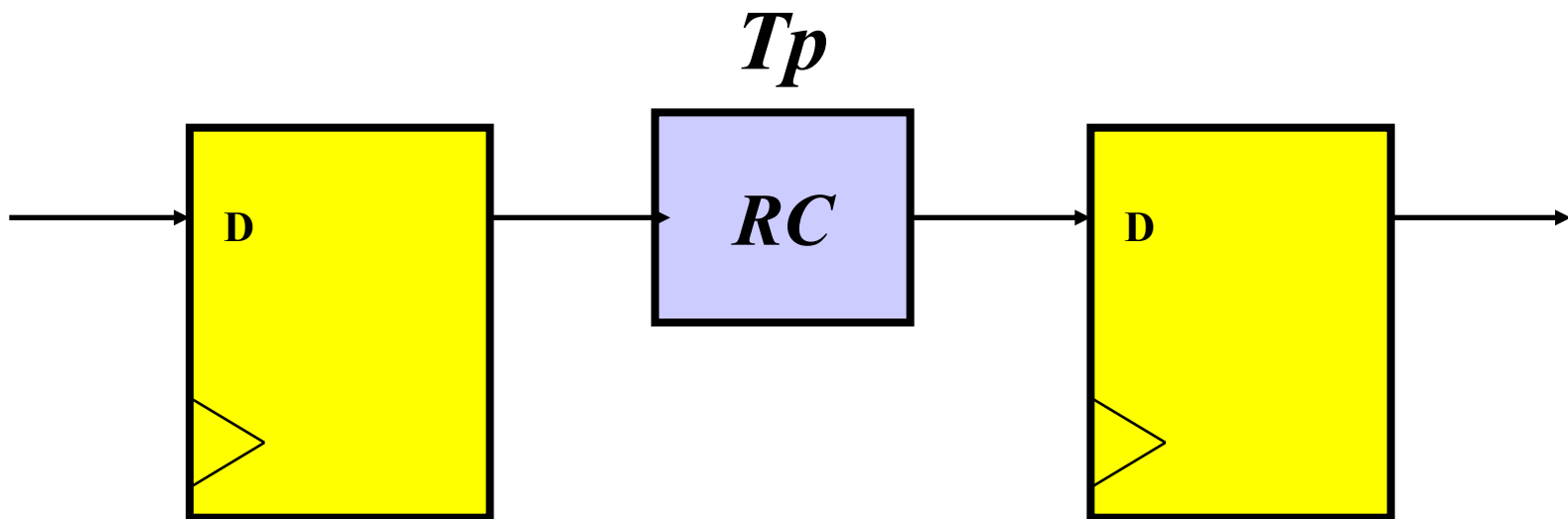


Overhead introdotto dai Pipeline Registers:

$$T_{clk} = T_d + T_P + T_{su}$$

Diagram illustrating the components of the clock cycle time (T_{clk}):

- T_{clk} : Clock Cycle
- T_d : Ritardo registro a monte (Register delay upstream)
- T_P : Ritardo stadio combinatorio più lento (Slowest combinatorial stage delay)
- T_{su} : Set-up registro a valle (Register set-up downstream)



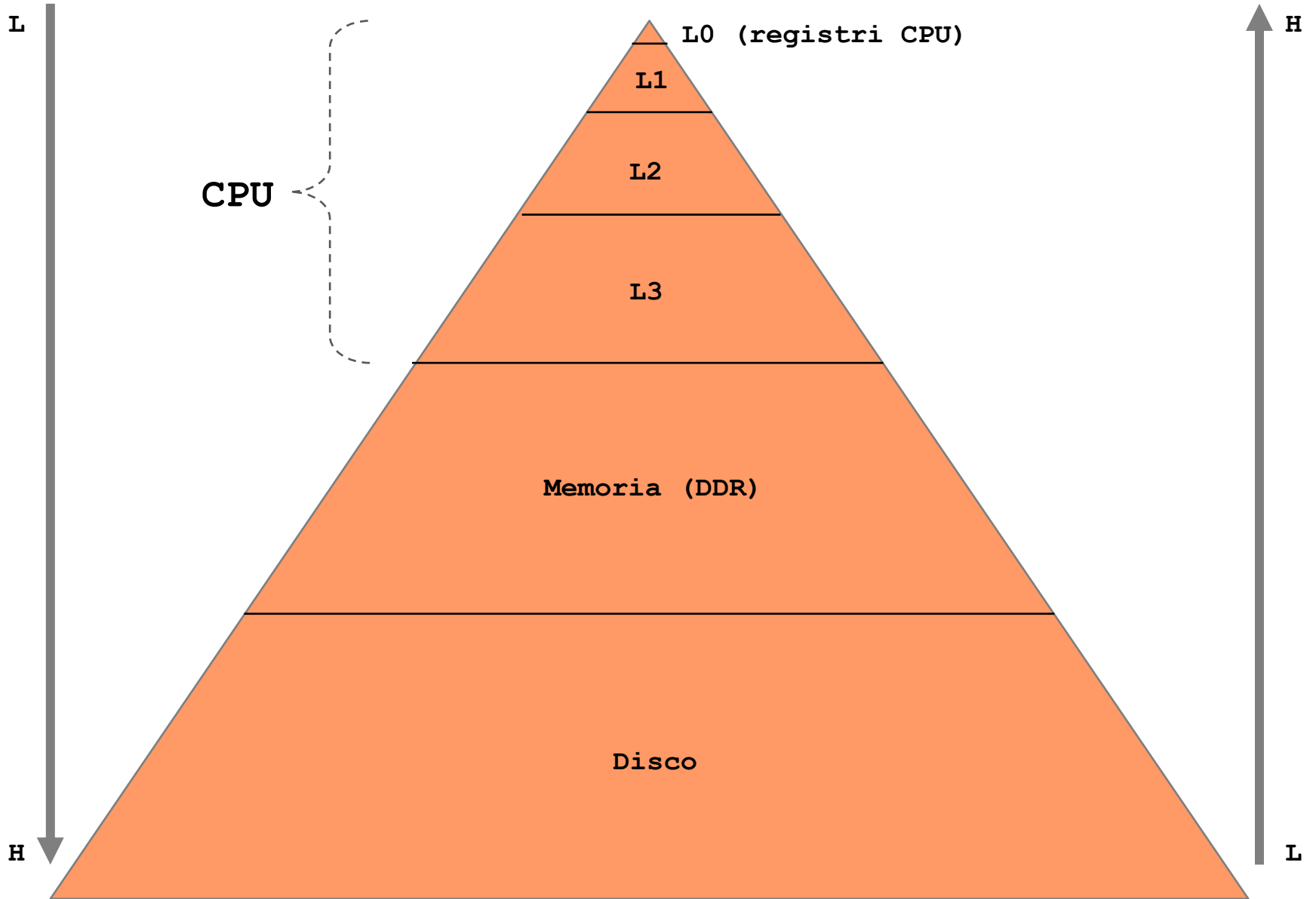
Requisiti per l'implementazione in pipeline

- Ogni stadio deve essere attivo in ogni ciclo di clock.
- E' necessario incrementare il PC in IF (invece che in ID).
- E' necessario introdurre un ADDER ($PC \leftarrow PC+4$ – $PC \leftarrow PC+1$) nello stadio IF.
- Sono necessari due MDR (che chiameremo LMDR e SMDR) per gestire il caso di una LOAD seguita immediatamente da una STORE (WB-MEM sovrapposti – sovrapposizione di due dati in attesa di essere scritti, uno in memoria e l'altro nel RF).
- In ogni ciclo di clock devono poter essere eseguiti 2 accessi alla memoria (IF, MEM): Instruction Memory (IM) e Data Memory (DM)
-> *Architettura 'Harvard'*
- Il clock della CPU è determinato dallo stadio più lento: IM, DM devono essere delle memorie *cache* (on-chip)
- I Pipeline Registers trasportano sia dati sia informazioni di controllo (l'unità di controllo è '*distribuita*' fra gli stadi della pipeline)

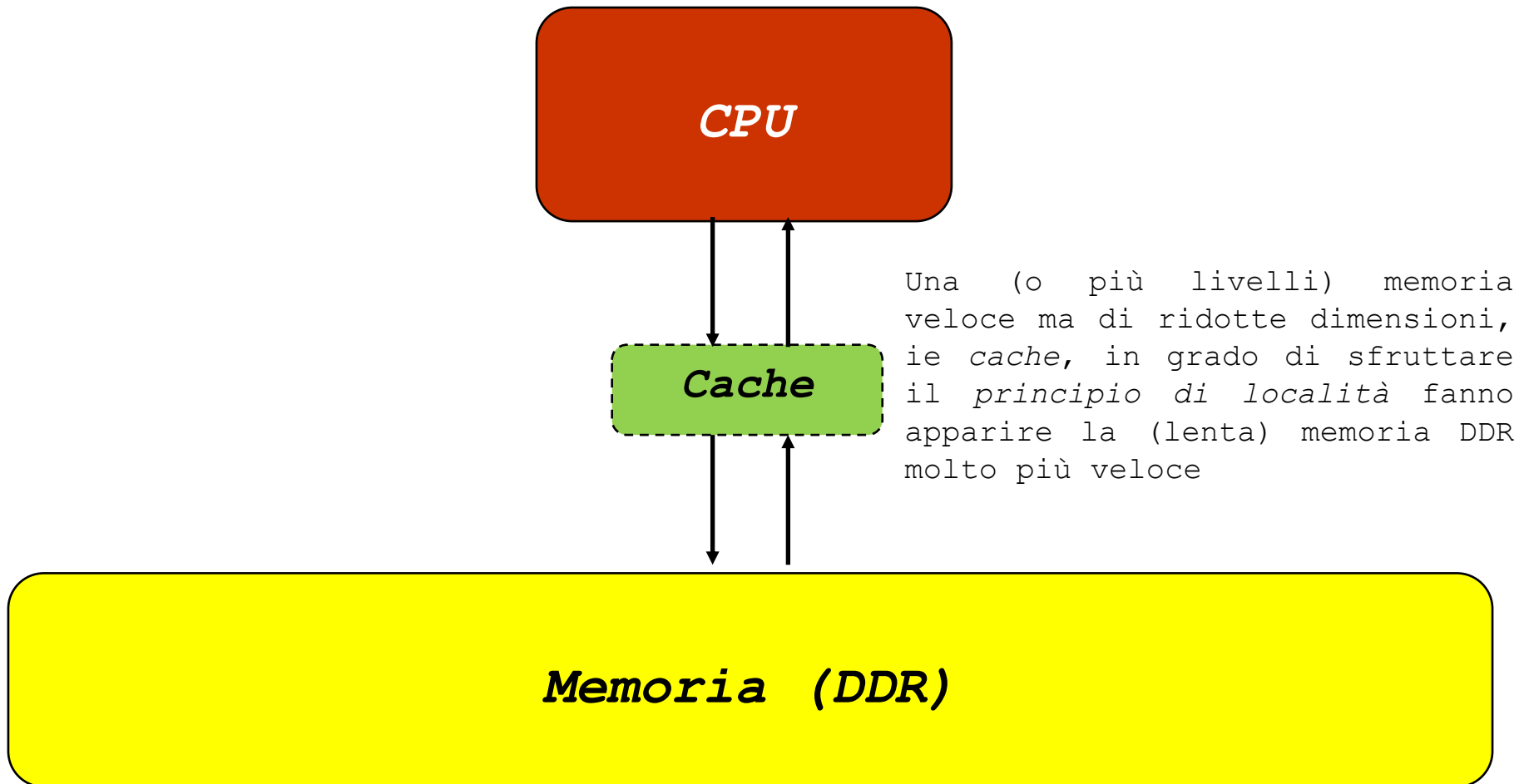
Gerarchia della memoria

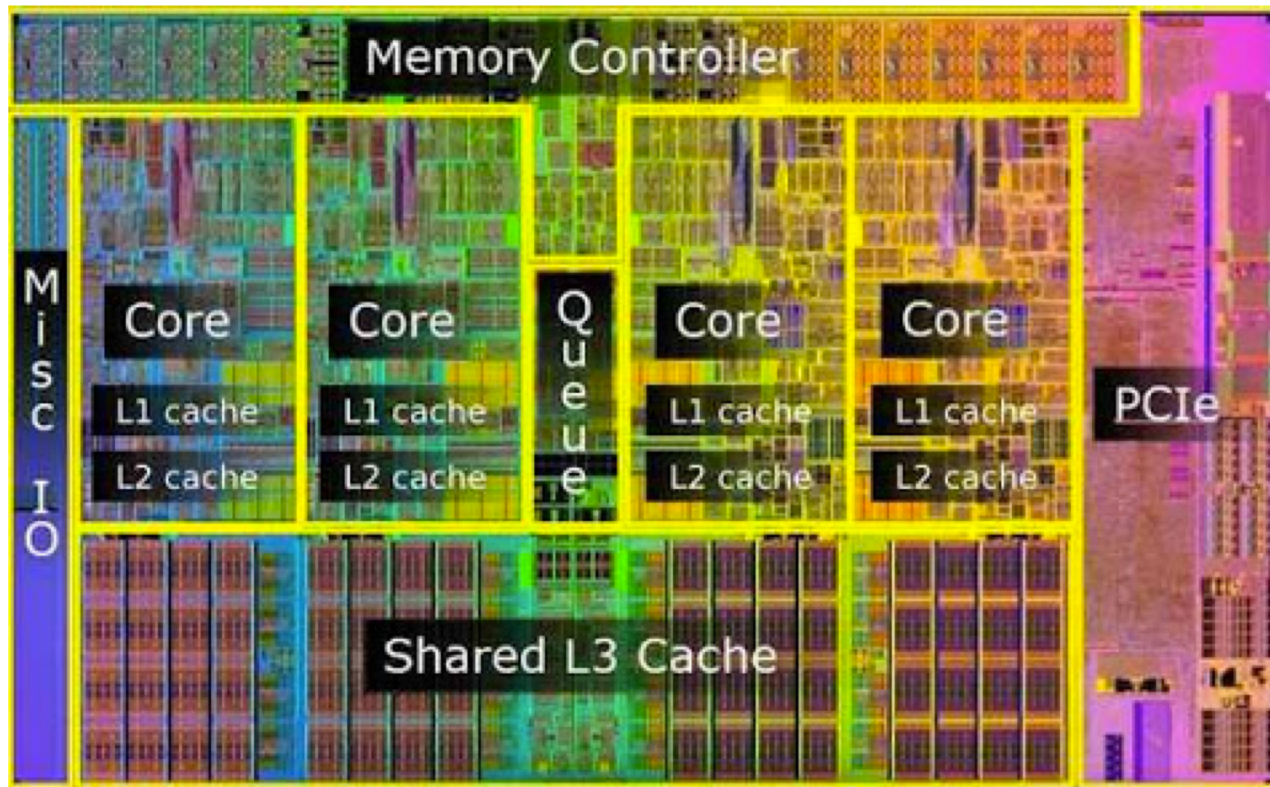
Tempo di accesso

Costo/Byte



Memorie cache





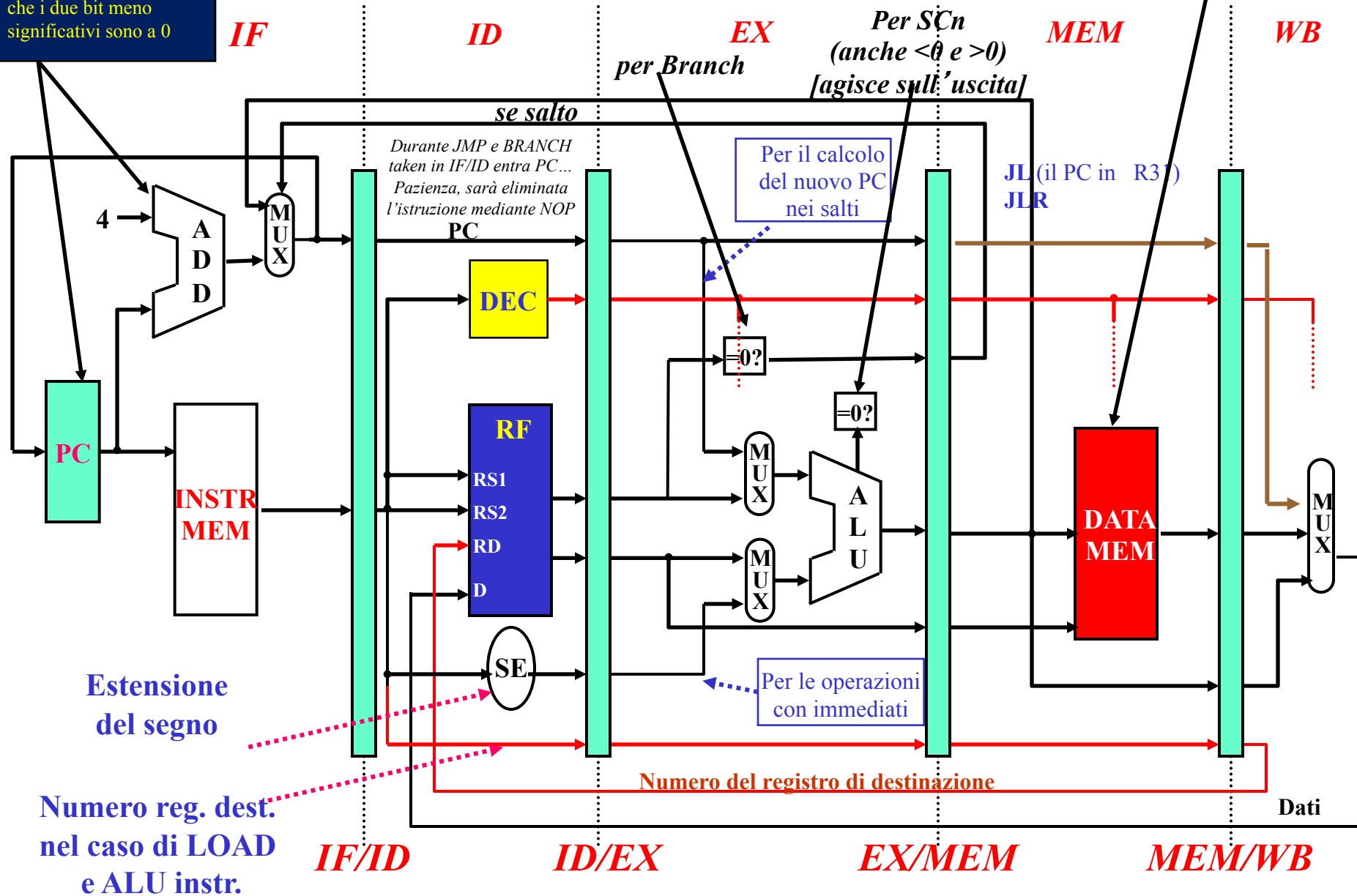
Area di silicio occupata da cache L1,L2,L3 in un Intel Core i5

Fonte: <https://thecodeartist.blogspot.com/2011/12/why-readmostly-does-not-work-as-it.html>

Datapath in Pipeline del DLX

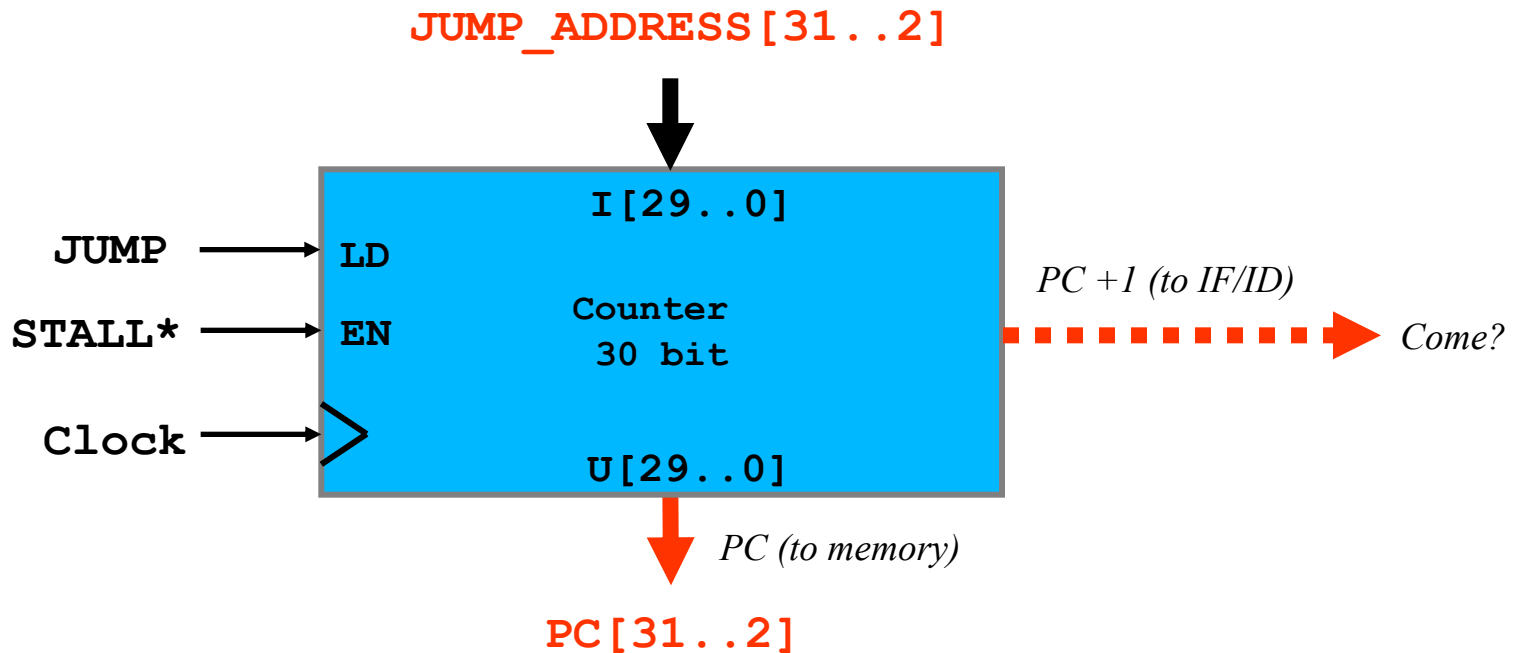
Contiene anche i circuiti di swap

In realtà è un contatore programmabile visto che i due bit meno significativi sono a 0



Stadio di Fetch con contatore 1/2

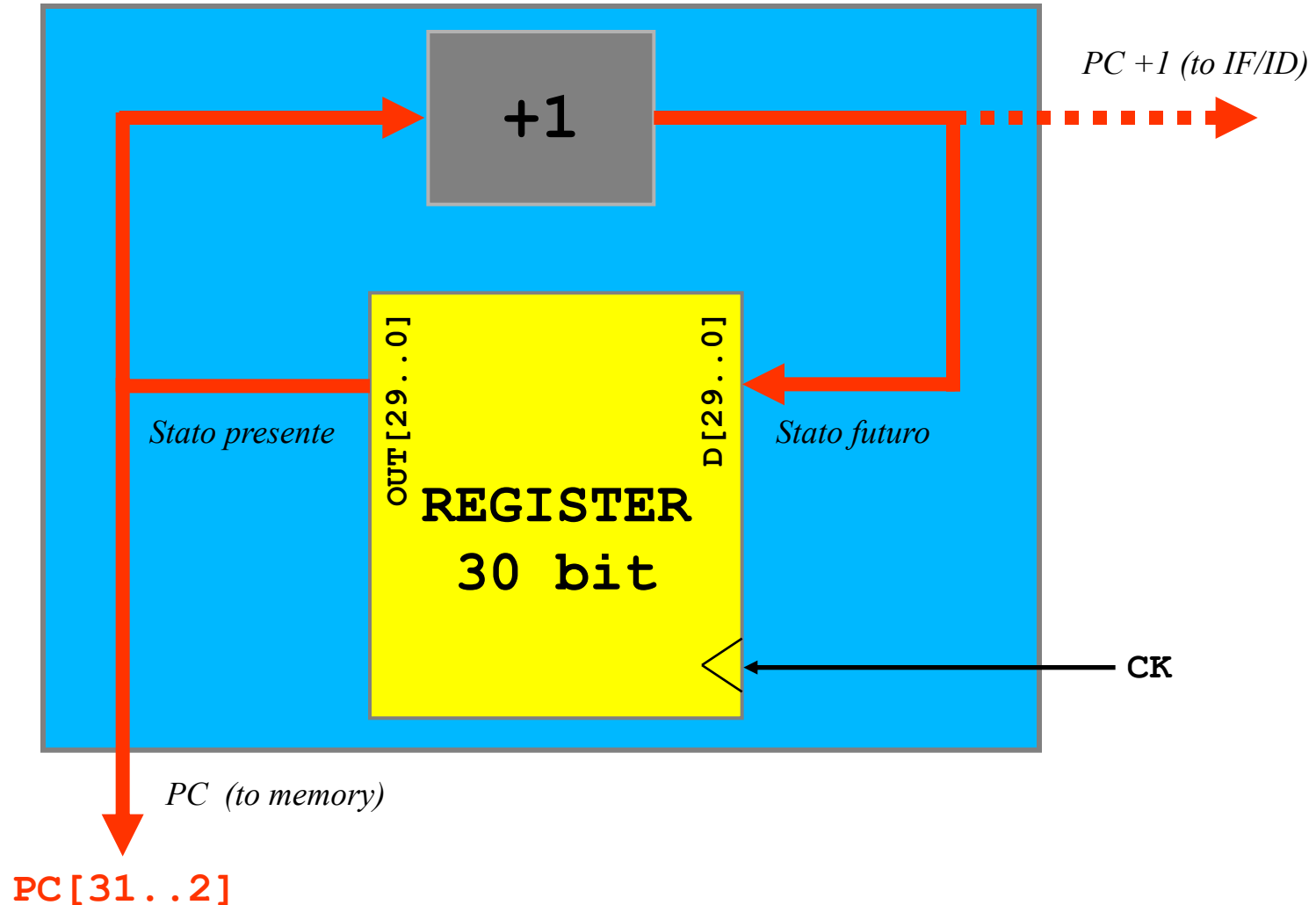
Con riferimento al primo schema del DLX *pipelined* studiato durante il corso (ma considerazioni analoghe si applicano alle altre versioni del DLX), la rete seguente consente di sostituire lo schema basato su registro e multiplexer con un contatore a 30 bit (i due bit meno significativi dell'indirizzo sono superflui perché il DLX esegue il fetch sempre a indirizzi allineati).



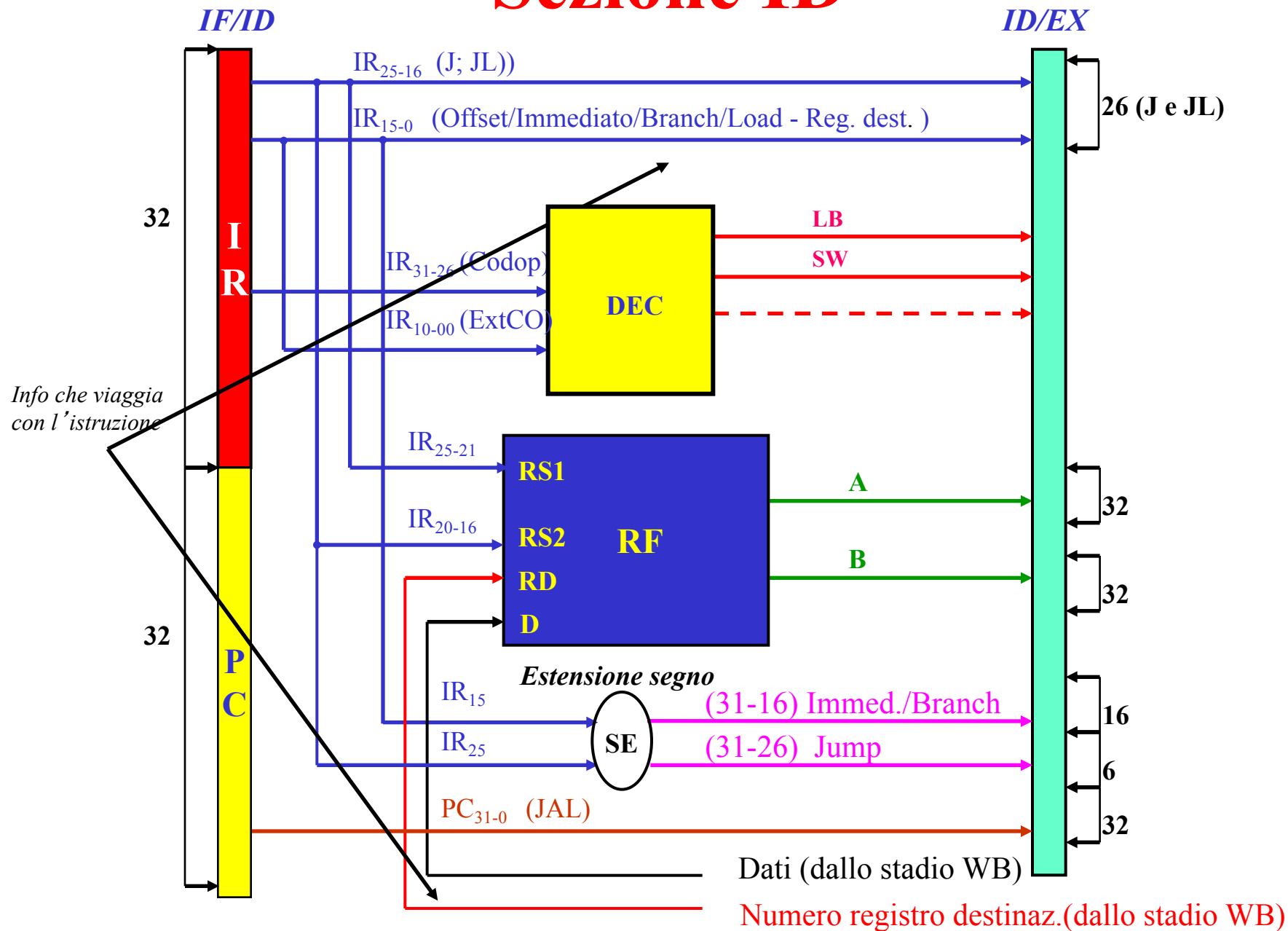
Sempre con riferimento allo stesso schema del DLX. Il segnale **JUMP** codifica se il DLX deve saltare alla destinazione specificata da **JUMP_ADDRESS[31..2]**. Entrambi i segnali sono inviati dallo stadio **MEM**. Il segnale **STALL**, è generato dalla Unità di Controllo quando lo stadio di **IF** deve essere bloccato.

Stadio di Fetch con contatore 2/2

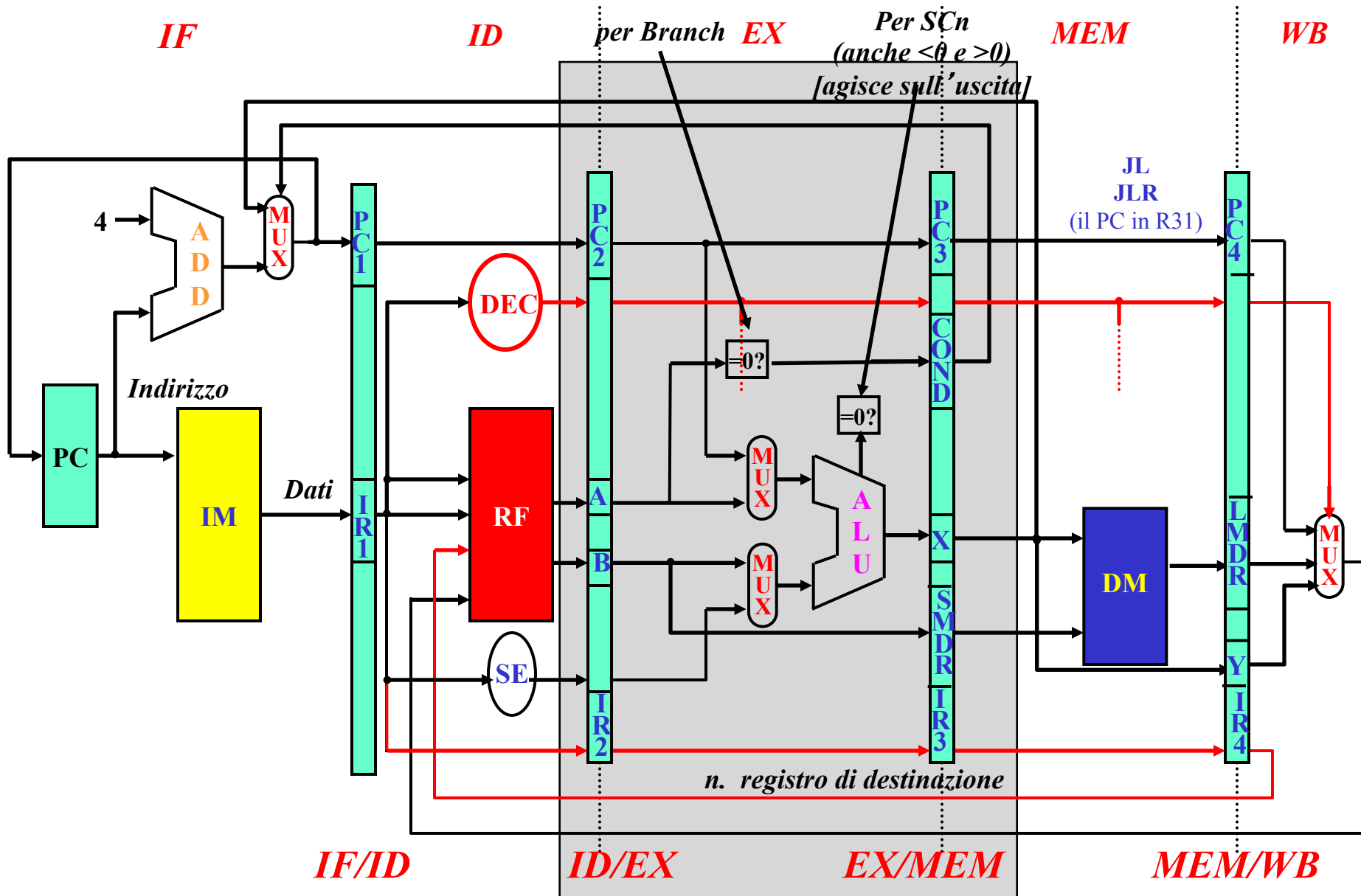
Un'osservazione: come possiamo generare $PC + 1$ per lo stadio IF/ID (quando viene eseguito il fetch a PC è necessario fare entrare nella pipeline (stadio IF/ID) $PC + 1$?



Sezione ID



Datapath in Pipeline del DLX



Esecuzione in pipeline di istruzione “ALU”

NB in questa come nelle altre istruzioni RD (RS2) è trasferita fino allo stadio WB

IF	$IR \leftarrow M[PC] ; PC \leftarrow PC + 4 ; PC1 \leftarrow PC + 4$
ID	$A \leftarrow RS1; B \leftarrow RS2; PC2 \leftarrow PC1; IR2 \leftarrow IR1$ $ID/EX \leftarrow \text{Decodifica istruzione};$
EX	$X \leftarrow A \text{ op } B$ $[PC3 \leftarrow PC2]$ $[IR3 \leftarrow IR2]$ oppure $X \leftarrow A \text{ op } (IR2_{15})^{16} \text{ ## } IR2_{15..0}$
MEM	$Y \leftarrow X$ (“parcheggio” in attesa di WB) $[IR4 \leftarrow IR3]$ $[PC4 \leftarrow PC3]$
WB	$RD \leftarrow Y$

N.B. al passare degli stadi IR perde i bit che non servono più in tutte le istruzioni. Da uno stadio al successivo vengono mantenuti i bit che servono qualunque sia l'istruzione

La decodifica attraversa tutti gli stadi

X : “ALUOUTPUT” (in EX/MEM), Y : “ALUOUTPUT1”

Esecuzione in pipeline di istruzione “MEM”

IF	$IR \leftarrow M[PC] ; PC \leftarrow PC + 4 ; PC1 \leftarrow PC + 4$
ID	$A \leftarrow RS1 ; B \leftarrow RS2 ; PC2 \leftarrow PC1 ; IR2 \leftarrow IR1$ $ID/EX \leftarrow \text{Decodifica istruzione};$
EX	$MAR \leftarrow A \text{ op } (IR2_{15})^{16} \text{ ## } IR2_{15..0}$ $SMDR \leftarrow B$ $\begin{matrix} IR3 \leftarrow IR2 \\ PC3 \leftarrow PC2 \end{matrix}$
MEM	$LMDR \leftarrow M[MAR] \text{ (LOAD)}$ oppure $PC4 \leftarrow PC3$ $M[MAR] \leftarrow SMDR \text{ (STORE)}$ $IR4 \leftarrow IR3$
WB	$RD \leftarrow LMDR \text{ (LOAD) [ext. Segno]}$

La decod.ifica
attraversa
tutti gli stadi



Esecuzione in pipeline di istruzione “BRANCH”

IF	$IR \leftarrow M[PC] ; PC \leftarrow PC + 4 ; PC1 \leftarrow PC + 4$
ID	$A \leftarrow RS1 ; B \leftarrow RS2 ; PC2 \leftarrow PC1 ; IR2 \leftarrow IR1$ $ID/EX \leftarrow \text{Decodifica istruzione};$
EX	$X \leftarrow PC2 \text{ op } (IR_{15})^{16} \text{ ## } IR_{15..0}$ $[PC3 \leftarrow PC2]$ $Cond \leftarrow A \text{ op } 0$ $[IR3 \leftarrow IR2]$
MEM	$\text{if } (Cond) PC \leftarrow X$ $[PC4 \leftarrow PC3]$ $[IR4 \leftarrow IR3]$
WB	(NOP)

La decodifica
attraversa
tutti gli stadi

X : “BTA (BRANCH TARGET ADDRESS)”

Il test avviene sul
valore del registro

Esecuzione in pipeline di un'istruzione "JR"

IF	IR $\leftarrow$ M[PC] ; PC $\leftarrow$ PC + 4 ; PC1 $\leftarrow$ PC + 4	
ID	A $\leftarrow$ RS1; B $\leftarrow$ RS2; PC2 $\leftarrow$ PC1; IR2$\leftarrow$IR1 ID/EX $\leftarrow$ Decodifica istruzione;;	
EX	X $\leftarrow$ A	[IR3 $\leftarrow$ IR2] [PC3 $\leftarrow$ PC2]
MEM	PC $\leftarrow$ X	[IR4 $\leftarrow$ IR3] [PC4 $\leftarrow$ PC3]
WB	(NOP)	

La decodifica
attraversa
tutti gli stadi

Come sarebbe la sequenza degli stati per una J ?

Esecuzione in pipeline di istruzione “JL o JLR”

I	IR <- M[PC] ; PC <- PC + 4 ; PC1 <- PC + 4
ID	A <- RS1; B <- RS2; PC2 <- P1; IR2<-IR1 ID/EX <- Decodifica istruzione;
EX	PC3 <- PC2 [IR3 <- IR2] X <- A (Se JLR) X <- PC2 + (IR₂₅)⁶ ## IR_{25..0} (Se JL)
MEM	PC <- X ; PC4<- PC3 [IR4 <- IR3]
WB	R31 <- PC4

*Evidenziati perché
in questo caso
utilizzati*

Decod. in
tutti gli stadi

**NB: La scrittura in R31 NON può essere anticipata perché
potrebbe sovrapporsi ad altra scrittura di registro**

Qual sarebbe la sequenza nel caso di SCN (ex SLT R1,R2,R3) ?

I F	IR <- M[PC] ; PC <- PC + 4 ; PC1 <- PC + 4
ID	A <- RS1; B <- RS2; PC2 <- P1; IR2<-IR1 ID/EX <- Decodifica istruzione;
EX	?
MEM	?
WB	?

Alee nelle Pipeline

Si verifica una situazione di “*Alea*” (“*Hazard*”) quando in un determinato ciclo di clock un’istruzione presente in uno stadio della pipeline non può essere eseguita in quel clock.

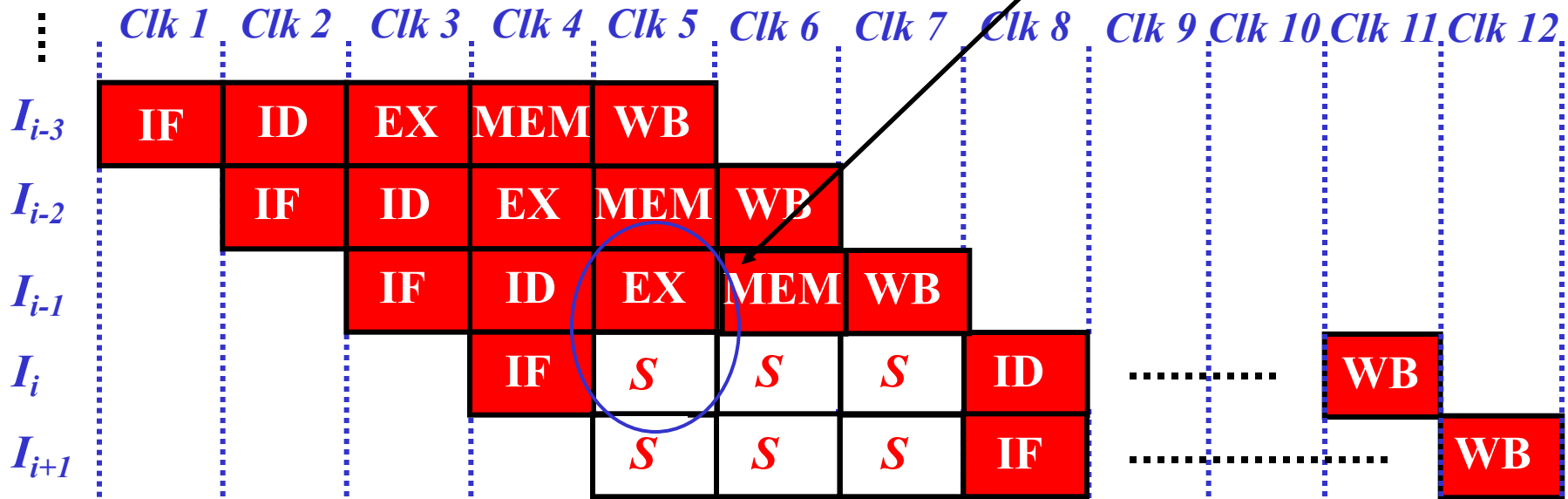
- ***Alee Strutturali*** - Una risorsa è condivisa fra due stadi della pipeline: le istruzioni che si trovano correntemente in tali stadi non possono essere eseguite simultaneamente.
- ***Alee di Dato*** – Sono dovute a *dipendenze* fra le istruzioni. Ad esempio una istruzione che legge un registro scritto da un’istruzione precedente (*RAW*).
- ***Alee di Controllo*** – Le istruzioni che seguono un branch *dipendono* dal risultato del branch (*taken/not taken*).



L’istruzione che non può essere eseguita viene bloccata (“stallo della pipeline”), insieme a tutte quelle che la seguono, mentre le istruzioni che la precedono avanzano normalmente (così da rimuovere la causa dell’alea).

Alee e Stalli

Effetto – ad esempio - di una alea di dato: se l'istruzione I_i necessita di un dato prodotto dalla istruzione I_{i-1} deve aspettare fino al WB della I_{i-1}



Stallo: blocco del clock dello stadio e di tutti quelli precedenti e propagazione progressiva agli stadi successivi

$$T_5 = 8 * CLK = (5 + 3) * CLK$$

$$T_N = N * 1 * CLK$$

$$T_5 = 5 * (1 + 3/5) * CLK$$

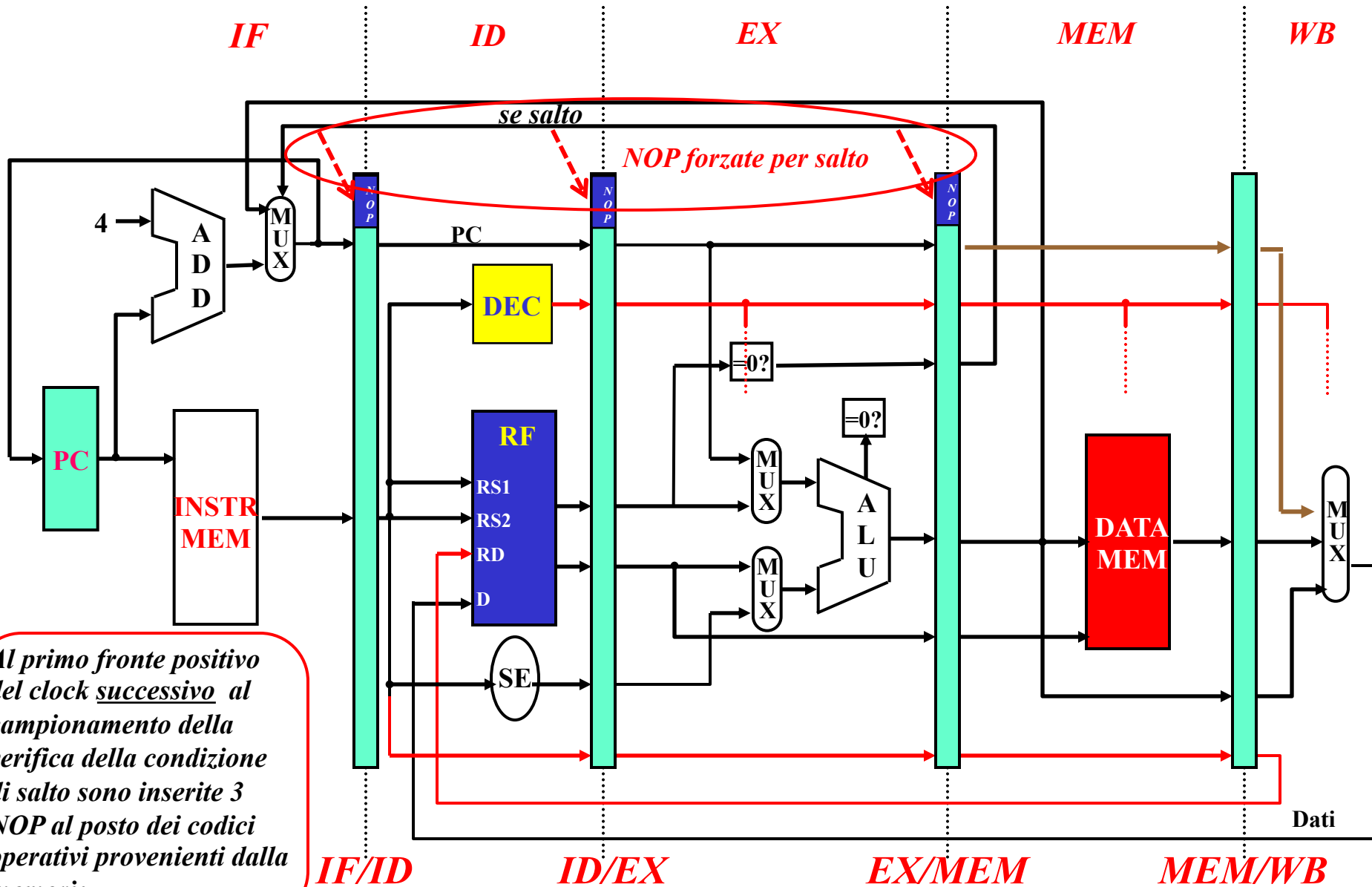
$$T_N = N * (1 + S) * CLK$$

CPI ideale

Stalli per istruzione

CPI effettivo

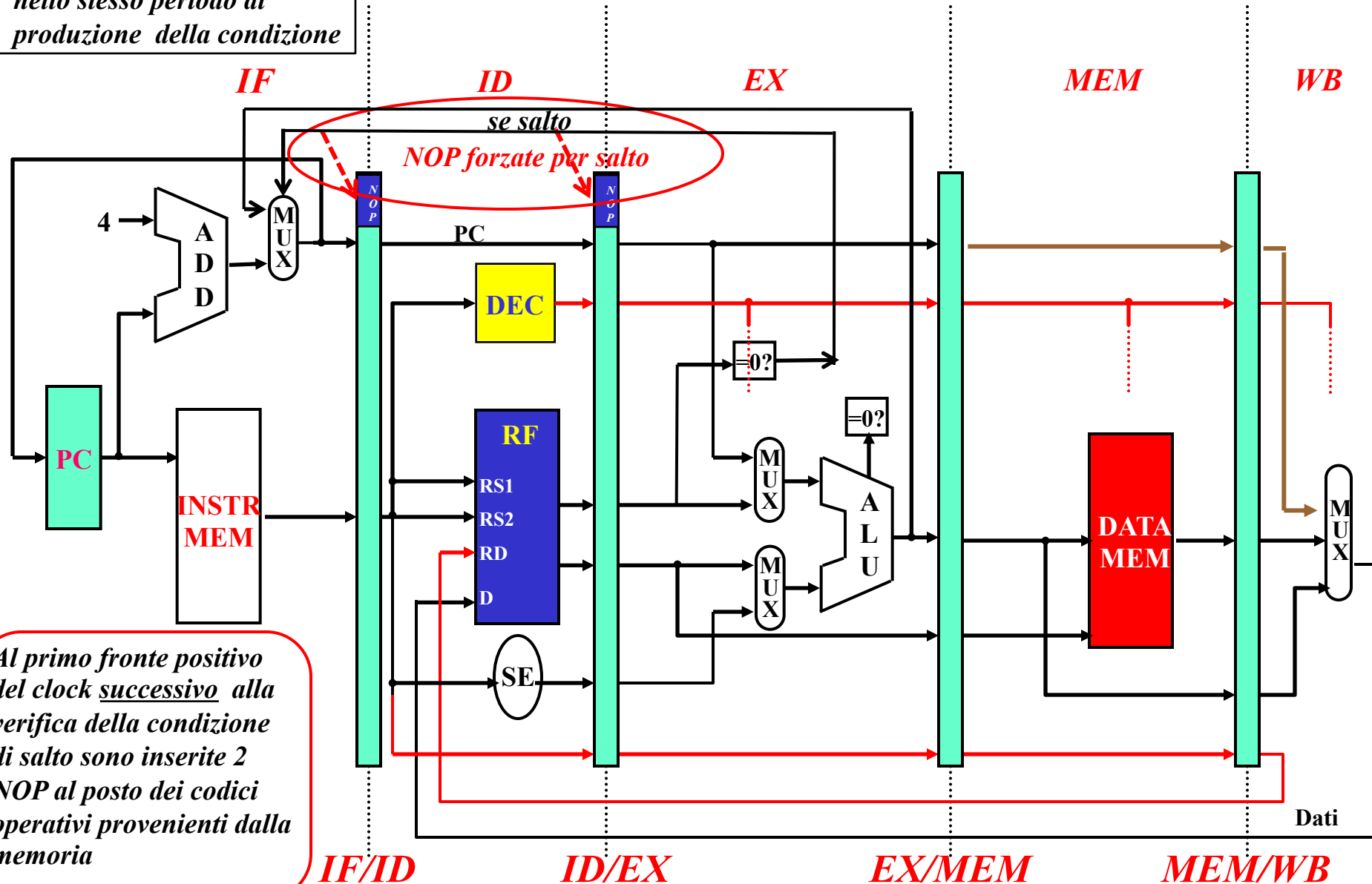
Stalli nel salto (1/3)



Al primo fronte positivo del clock successivo al campionamento della verifica della condizione di salto sono inserite 3 NOP al posto dei codici operativi provenienti dalla memoria

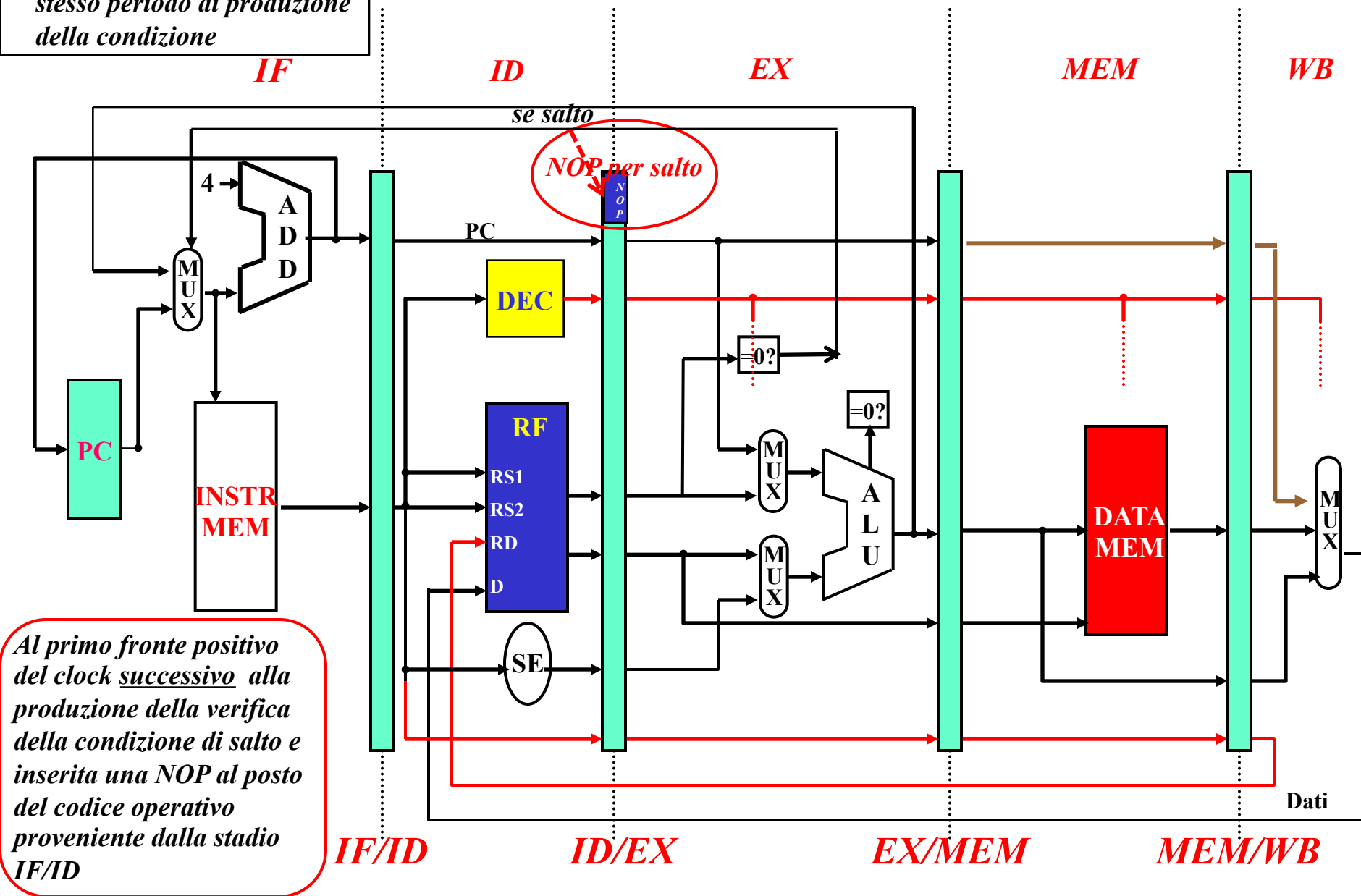
NB In questo caso la condizione di salto e il nuovo PC sono presentati al MUX nello stesso periodo di produzione della condizione

Stalli nel salto (2/3)

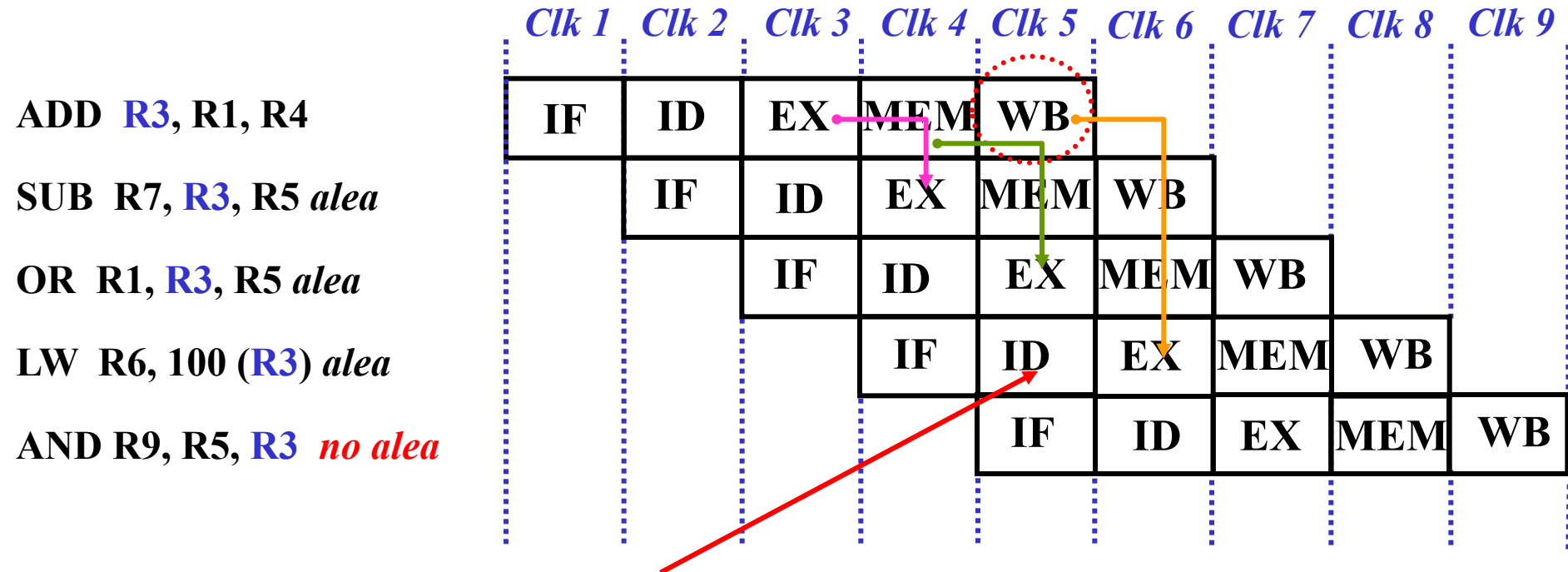


NB In questo caso la condizione di salto e il nuovo PC agiscono sul MUX nello stesso periodo di produzione della condizione

Stalli nel salto (3/3)



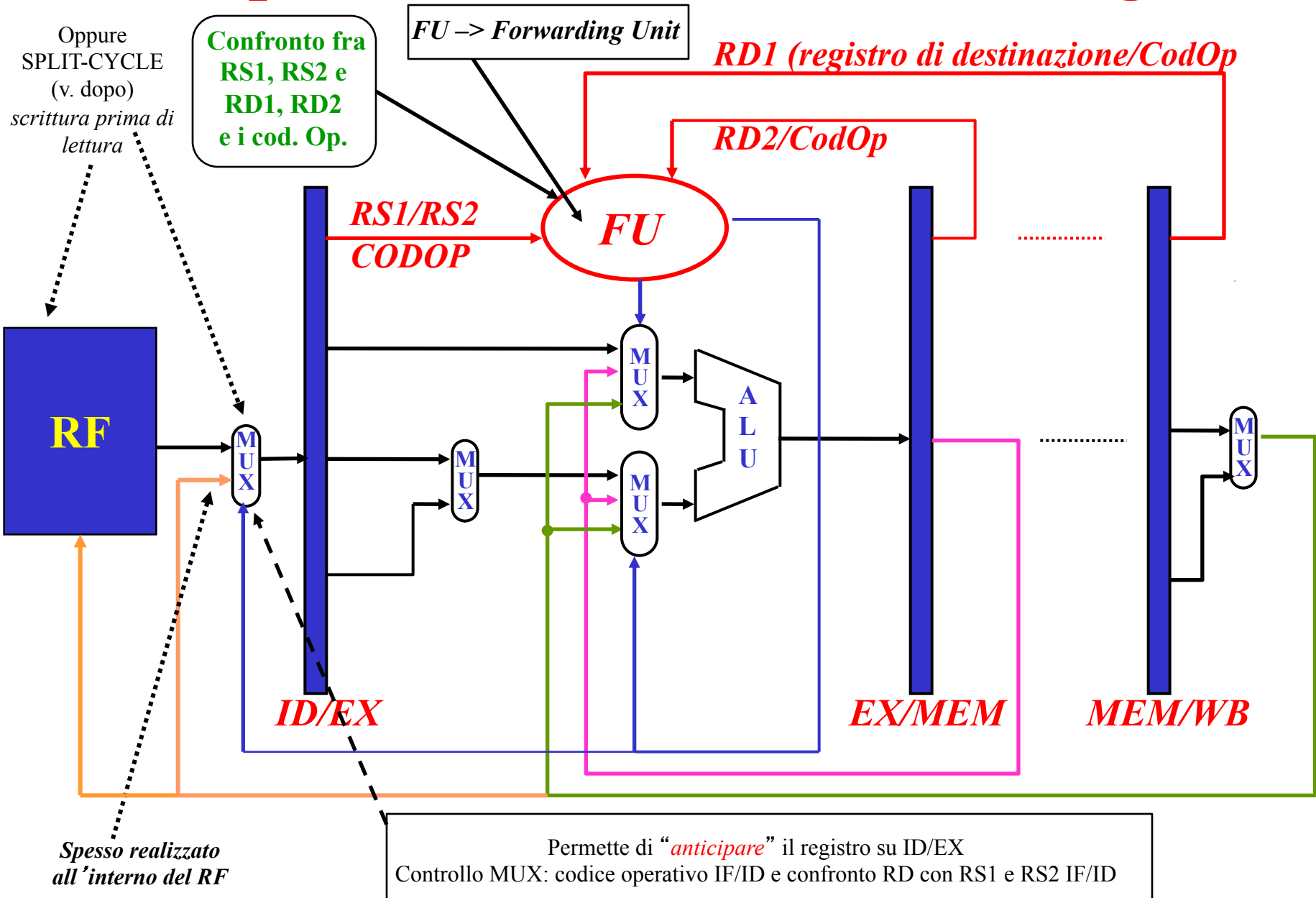
Forwarding



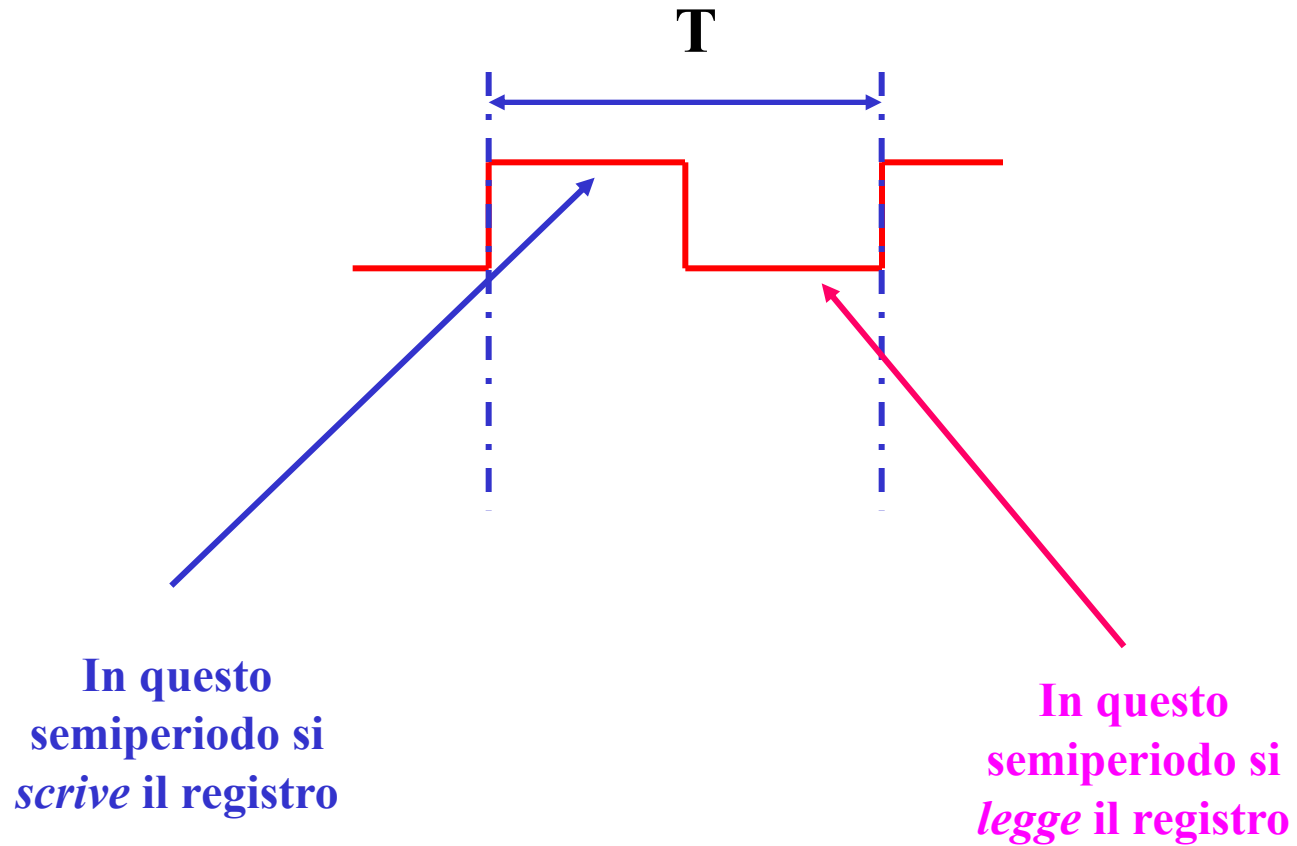
Anche qui il dato non è ancora in RF per essere estratto in ID !

Il *forwarding* consente di eliminare quasi tutte le alee di tipo RAW della pipeline del DLX *senza stallare* la pipeline.
(NB: nel DLX si *alterano* i registri *solo* in WB)

Implementazione del Forwarding



Split-cycle



Alea di dato dovuta alle istruzioni di LOAD

LW	R1,32(R6)	IF	ID	EX	MEM	WB
ADD	R4,R1,R7		IF	ID	EX	MEM
SUB	R5,R1,R8			IF	ID	EX
AND	R6,R1,R7				IF	ID

N.B. il dato richiesto dalla ADD è presente solo *alla fine* di MEM. L'alea non può essere eliminata con il forwarding (a meno di non aprire una ulteriore di ingresso ai mux della ALU dalla memoria – ritardi !)

*Di fatto non viene generato il clock.
Il blocco di un clock si propaga
lungo la pipeline uno stadio alla
volta.*



**E' necessario stallare la
pipeline**

*Dalla fine di
questo stadio in
poi normale
forwarding*

LW	R1,32(R6)	IF	ID	EX	MEM	WB
ADD	R4,R1,R7		IF	ID	S	EX MEM
SUB	R5,R1,R8			IF	S	ID EX
AND	R6,R1,R7				S	IF ID

Delayed load

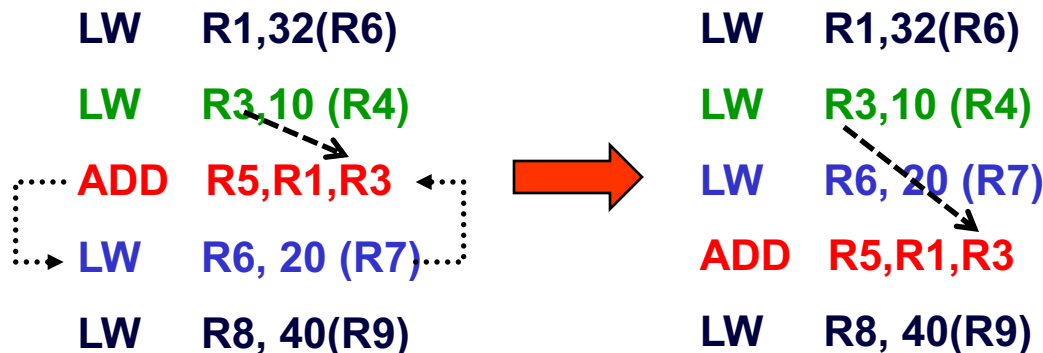
In diverse CPU RISC l'alea associata alla LOAD non è gestita in HW stallando la pipeline ma è gestita via SW dal compilatore (*delayed load*):

Istruzione LOAD

delay slot

Istruzione Successiva

Il compilatore cerca di riempire il delay-slot con un'istruzione "utile" (caso peggiore: NOP).



Alee di Controllo

Next Instruction

Address

PC

PC+4

PC+8

PC+12

*PC+4+200
(BTA)*

BEQZ R4, 200

SUB R7, R3, R5

OR R1, R3, R5

LW R6, 100 (R8)

⋮

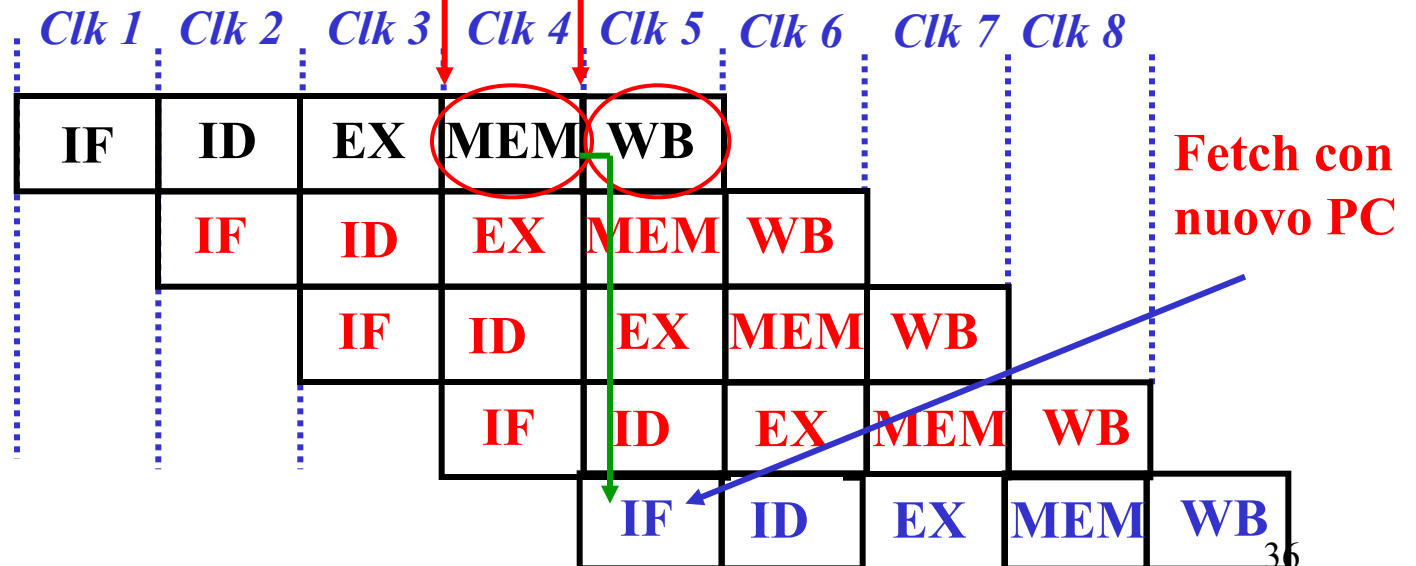
AND R9, R5, R3

R4 = 0 : *Branch Target Address*
(taken)

R4 ≠ 0 : *PC+4*
(not taken)

Nuovo valore PC calcolato (Aluout)

Nuovo valore in PC (un clock dopo)



Datapath in Pipeline del DLX (caso 1/3) - (Branch o JMP)

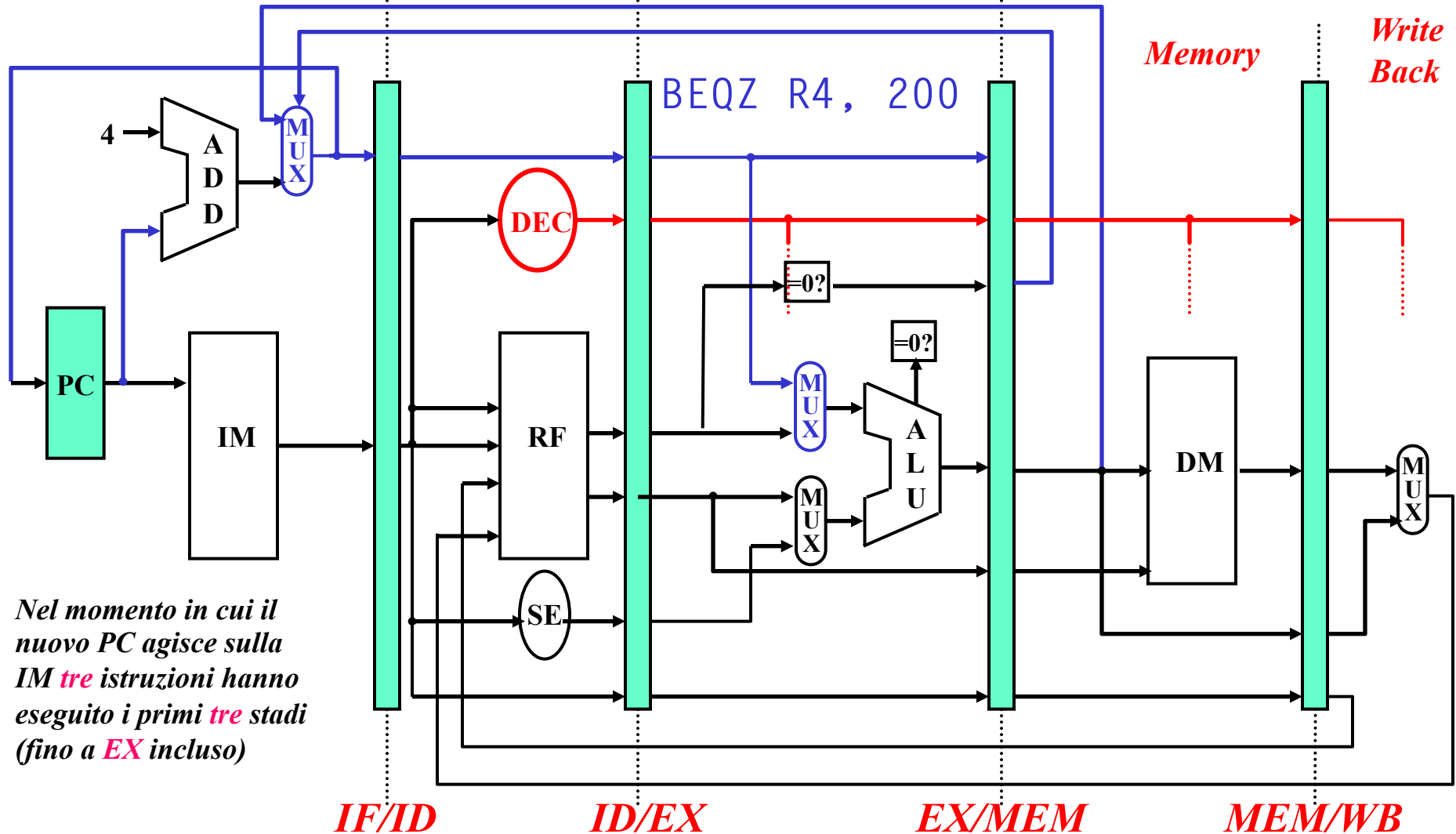
Instruction Fetch

Instruction Decode

Execute

Memory

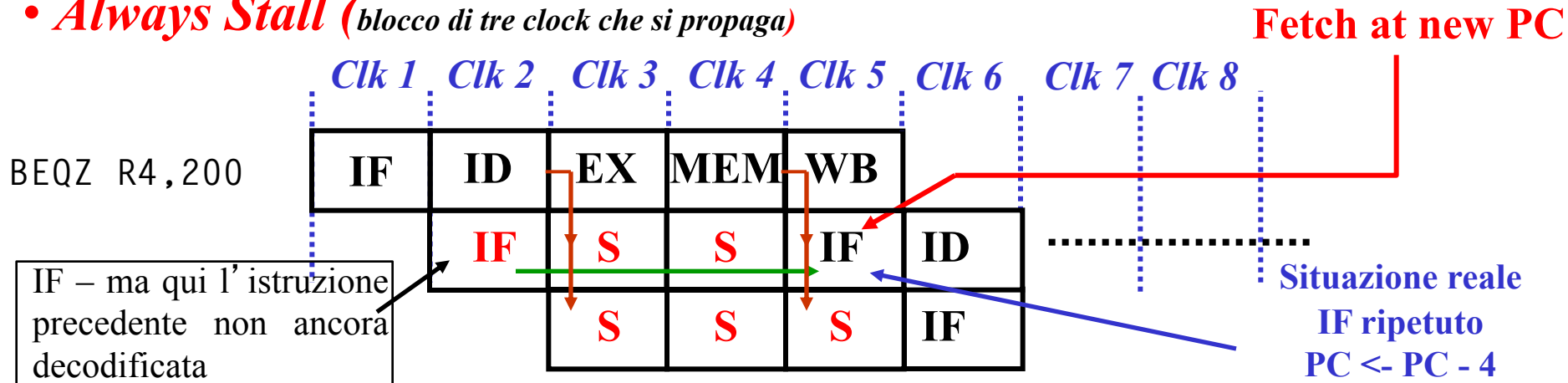
Write Back



*Nel momento in cui il nuovo PC agisce sulla IM **tre** istruzioni hanno eseguito i primi **tre** stadi (fino a **EX** incluso)*

Gestione delle Alee di Controllo

- *Always Stall* (blocco di tre clock che si propaga)

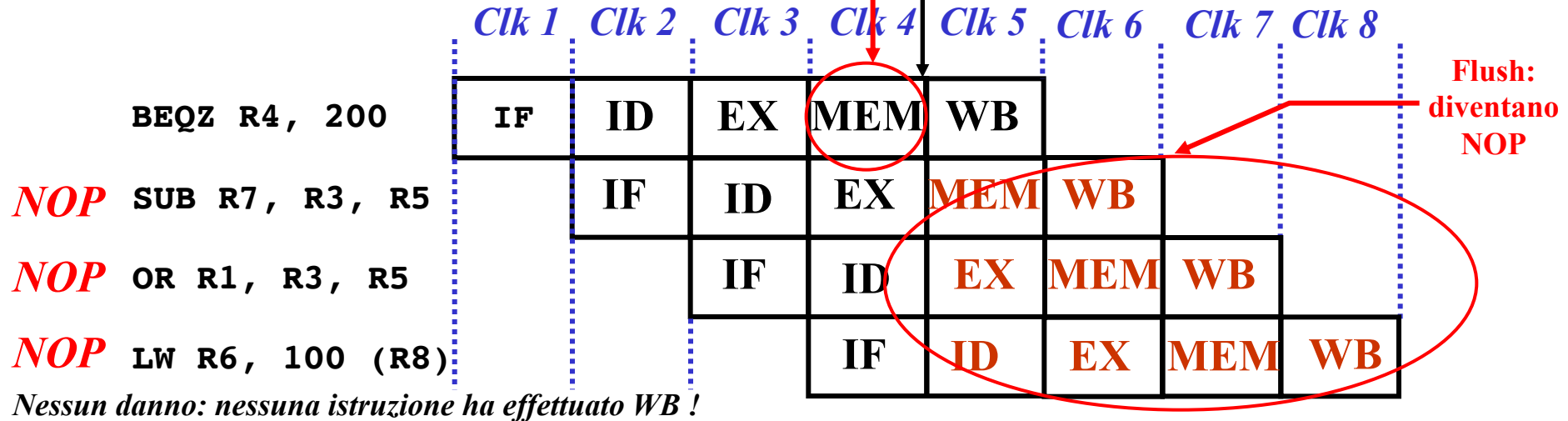


Hyp.: Freq.Branch = 25 %

$$\longrightarrow CPI = (1 + S) = (1 + 3 * 0.25) = 1.75$$

- *Predict Not Taken*

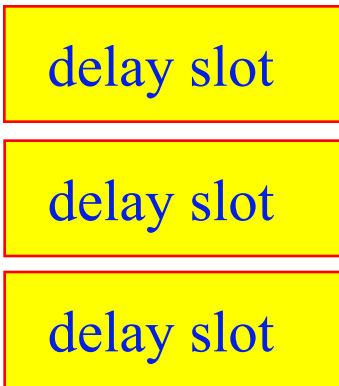
Qui il nuovo valore è campionato dal PC



Delayed branch

Similmente al caso della LOAD, in diverse CPU di tipo RISC l'alea associata alle istruzioni di BRANCH è gestita via SW dal compilatore (*delayed branch*):

Istruzione BRANCH



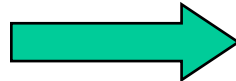
Il compilatore cerca di riempire i delay-slot con istruzioni “utili” (caso peggiore: NOP).

Istruzione Successiva

Delayed branch/jump

Originale

Add R5, R4, R3
Sub R6, R5, R2
Or R14, R6, R21
Sne R1, R8, R9 ; condizione di branch
Br R1, +100



Compilato

Sne R1, R8, R9 ; condizione di branch
Br R1, +100
Add R5, R4, R3
Sub R6, R5, R2
Or R14, R6, R21

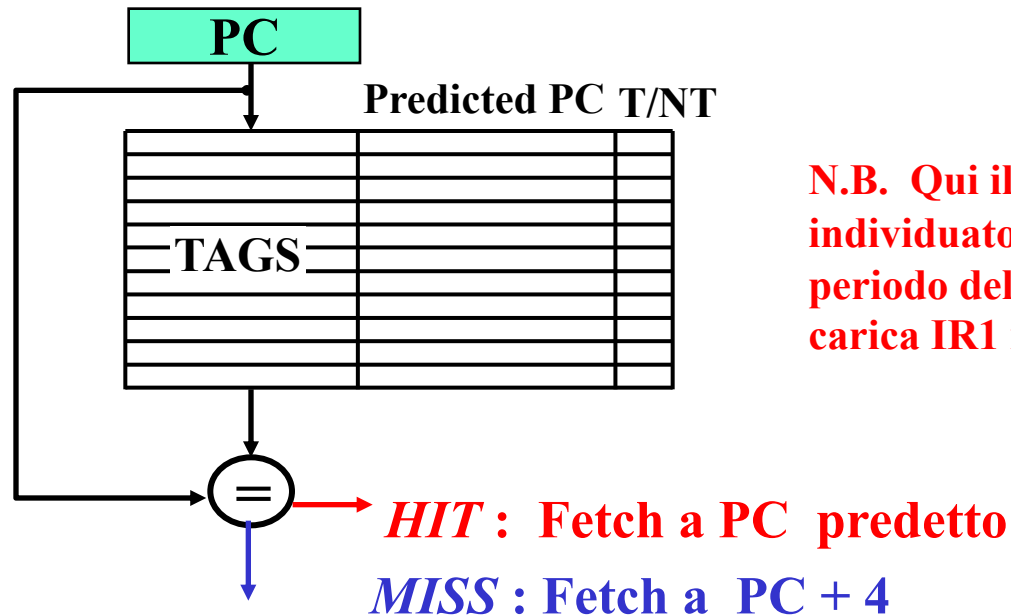
*Ovviamente in questo
gruppo non debbono
esserci salti !!!!*

*Eseguite in
entrambi i casi*

Al posto di una o più istruzioni “posposte” il compilatore mette delle NOP in caso non riesca a trovarne di adatte

Gestione delle Alee di Controllo con BTB

Dynamic Prediction: Branch Target Buffer -> nessuno stallo (quasi)

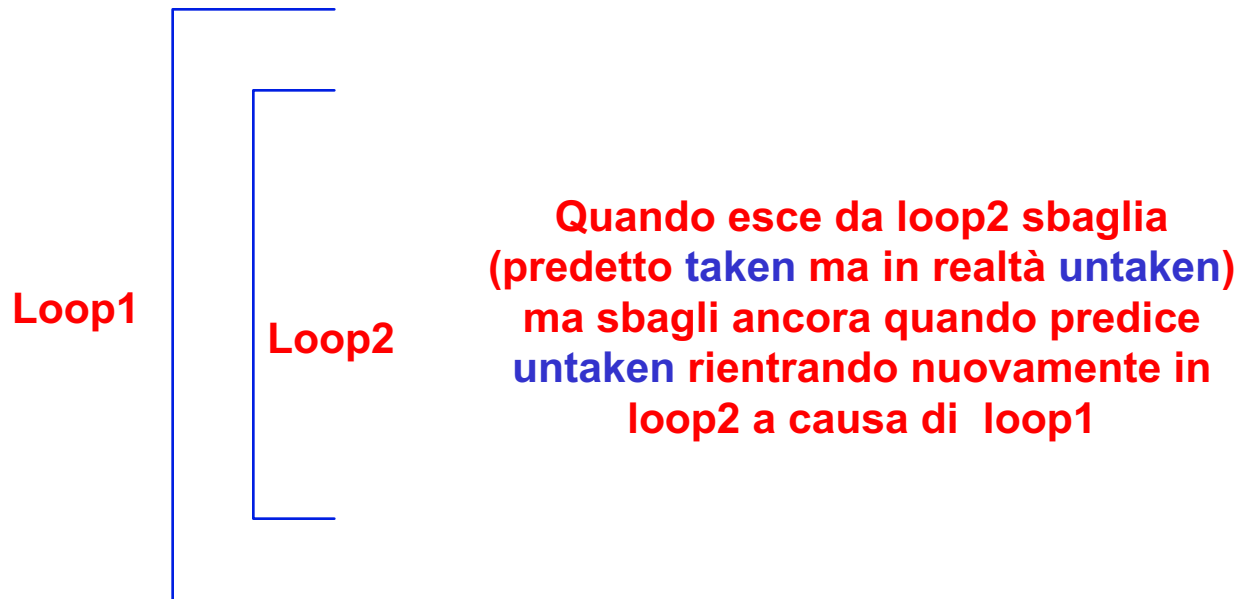


N.B. Qui il branch è individuato *durante* il periodo del clock IF che carica IR1 in IF/ID

Predizione Corretta : 0 stalli

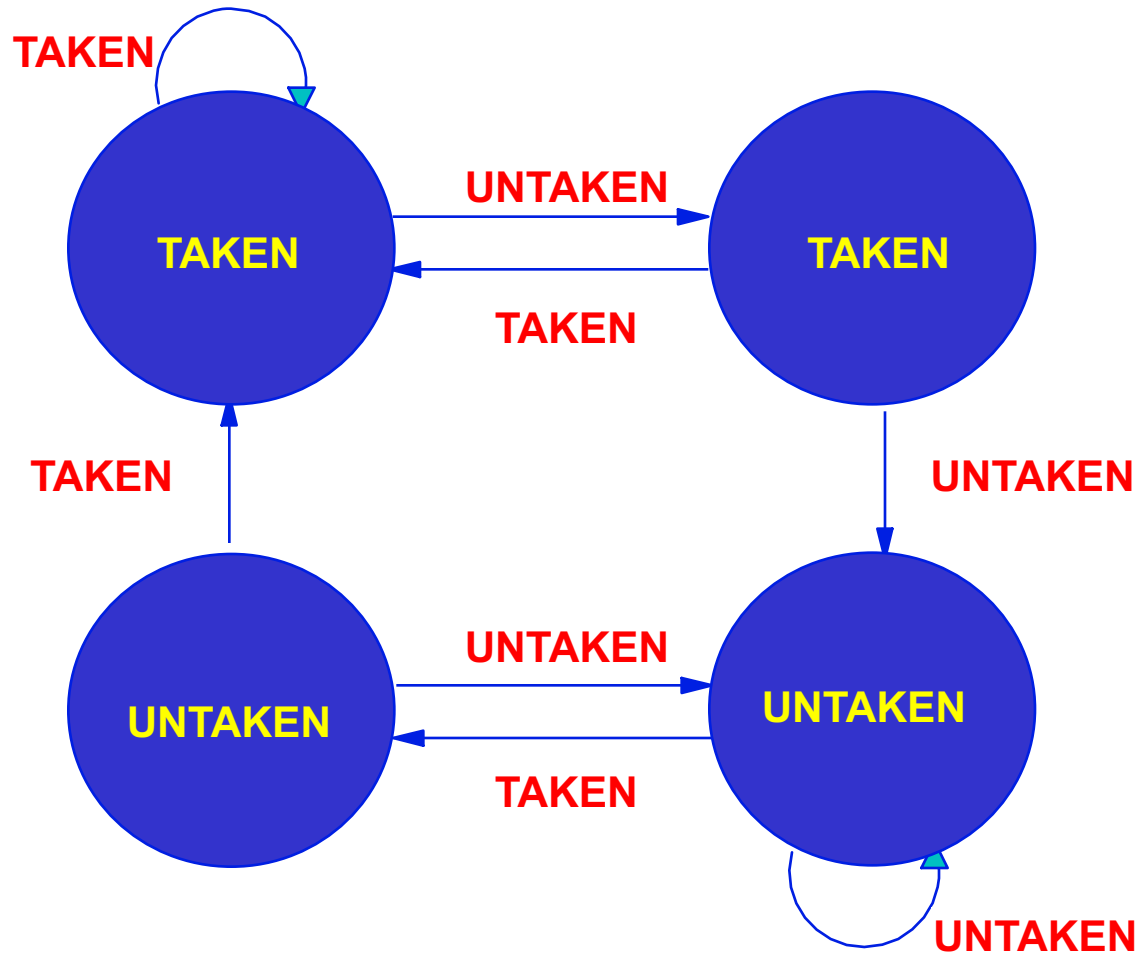
Predizione Errata : da 1 a 3 stalli (fetch corretto in ID o EX v. precedentemente)

Buffer di predizione: caso più semplice un bit che indica cosa è successo l'ultima volta.



In presenza di preponderanza di un caso quando si verifica il caso opposto si hanno *due* errori successivi.

Normalmente *due* bits.



Esempio, molto frequente, di loop annidato:

```
for (i=0; i<5000; i++)  
  for (j=0; j<1000; j++)  
  {  
      x[i,j] = i*j + i + j;  
      ...  
      ...  
  }
```